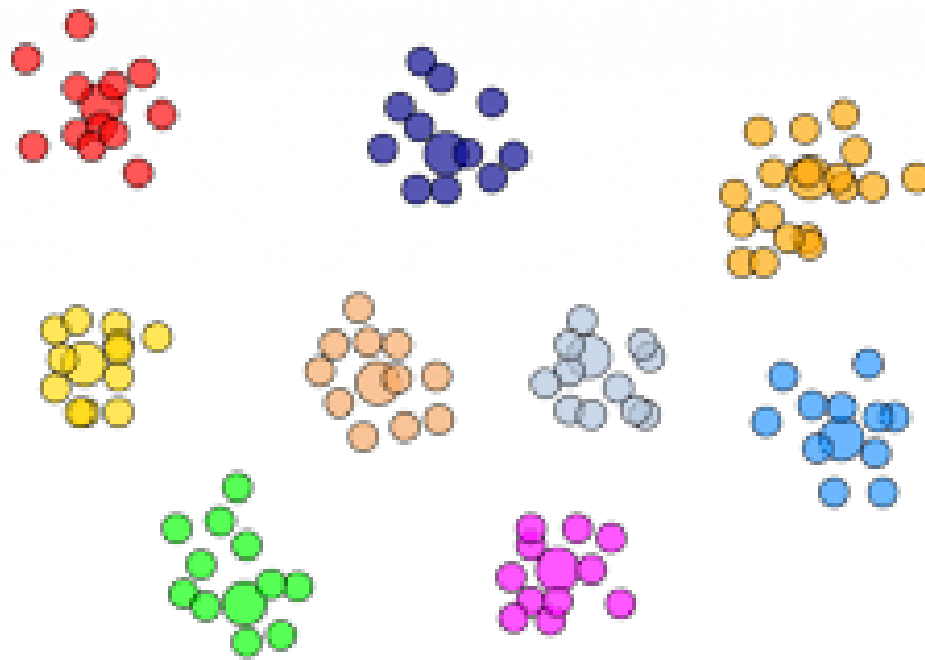
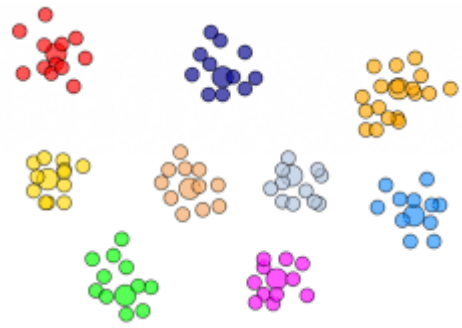


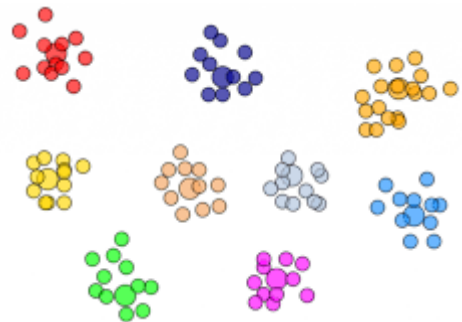


Clustering III



K-Medoids

- En el **k-medoids** es un algoritmo de agrupamiento relacionado con el algoritmo k-means.
- Tanto el **k-medoids** como el **k-means** son algoritmos que trabajan con particiones (dividiendo el conjunto de datos en grupos) y ambos intentan minimizar la distancia entre puntos que se añadirían a un grupo y otro punto designado como el centro de ese grupo.
- En contraste con el algoritmo k-means, k-medoids escoge datapoints como centros y trabaja con una métrica arbitraria de distancias entre datapoints.



K-Medoids

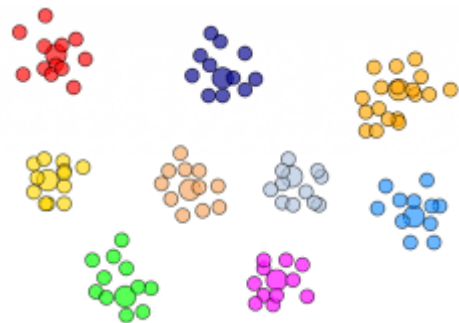
K-Means

Asigna puntos al “centro” más cercano y luego recalcula cada centro como el promedio de sus puntos (centroide).

Minimiza suma de distancias al cuadrado.

K-Medoids

Igual, asigna puntos al “centro” más cercano, pero el centro debe ser un punto real del dataset (medoide), elegido para minimizar la suma de distancias dentro del cluster.



K-Medoids: ejemplo

K-Means

Dataset ($k = 2$)

Dos grupos claros y un outlier cerca del grupo A pero algo lejos:

Grupo A (compacto):

- A1 (1,1)
- A2 (1,2)
- A3 (2,1)
- A4 (2,2)

Outlier:

- O (6,1)

Grupo B (compacto):

- B1 (8,8)
- B2 (8,9)
- B3 (9,8)
- B4 (9,9)

Paso A — Asignación (intuición)

Con $k=2$, la separación natural queda:

- Cluster 1: A1,A2,A3,A4 y el outlier O (porque O está mucho más cerca de A que de B)
- Cluster 2: B1,B2,B3,B4

Paso B — Recalcular centroides (promedios)

Centroide del Cluster 1 (A + outlier):

- $\bar{x} = (1 + 1 + 2 + 2 + 6)/5 = 12/5 = 2.4$
- $\bar{y} = (1 + 2 + 1 + 2 + 1)/5 = 7/5 = 1.4$

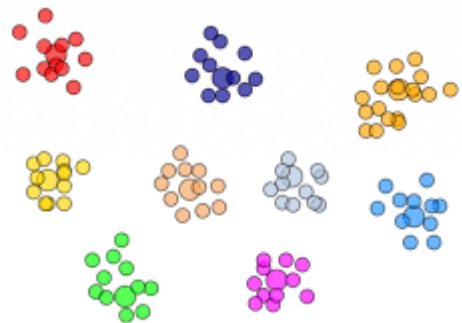
✓ C1 = (2.4, 1.4) (nota: no es un punto real del dataset)

Centroide del Cluster 2 (B):

- $\bar{x} = (8 + 8 + 9 + 9)/4 = 34/4 = 8.5$
- $\bar{y} = (8 + 9 + 8 + 9)/4 = 34/4 = 8.5$

✓ C2 = (8.5, 8.5)

Observación clave: el outlier arrastra el centroide del cluster A: sin el outlier, el centro “natural” del grupo A sería alrededor de (1.5, 1.5), pero pasa a (2.4, 1.4).



K-Medoids: ejemplo

K-Medoids

Dataset ($k = 2$)

Dos grupos claros y un outlier cerca del grupo A pero algo lejos

Grupo A (compacto):

- A1 (1,1)
- A2 (1,2)
- A3 (2,1)
- A4 (2,2)

Outlier:

- O (6,1)

Grupo B (compacto):

- B1 (8,8)
- B2 (8,9)
- B3 (9,8)
- B4 (9,9)

Paso A — Misma asignación (en este dataset)

Normalmente la asignación queda igual que antes:

- Cluster 1: A1,A2,A3,A4,O
- Cluster 2: B1,B2,B3,B4

Paso B — Elegir el medoide (el “mejor representante” real)

En el Cluster 1, probamos candidatos entre sus propios puntos y elegimos el que minimiza la suma de distancias a los demás. Aquí el mejor sale ser:

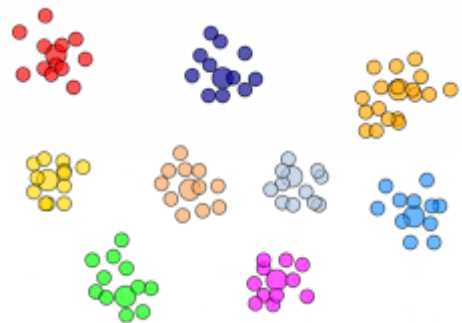
✓ M1 = (2,1) (A3)

El outlier (6,1) *nunca* será medoide porque tiene una suma de distancias enorme al resto.

En el Cluster 2, por simetría cualquiera de los cuatro puntos funciona igual; por ejemplo:

✓ M2 = (8,8) (B1)

Observación clave: el “centro” de K-Medoids no se desplaza hacia el outlier, porque está obligado a elegir un representante real y “central” del grupo.



K-Medoids: ejemplo

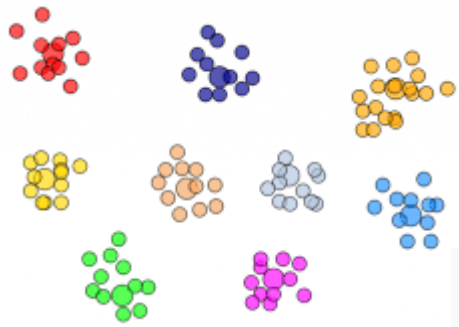
Resultado final comparado (la diferencia que importa)

- K-Means
- Centro Cluster 1: (2.4, 1.4) (promedio → sensible al outlier)
- Centro Cluster 2: (8.5, 8.5)
- K-Medoids
- Medoide Cluster 1: (2,1) (punto real → robusto)
- Medoide Cluster 2: (8,8) (punto real)

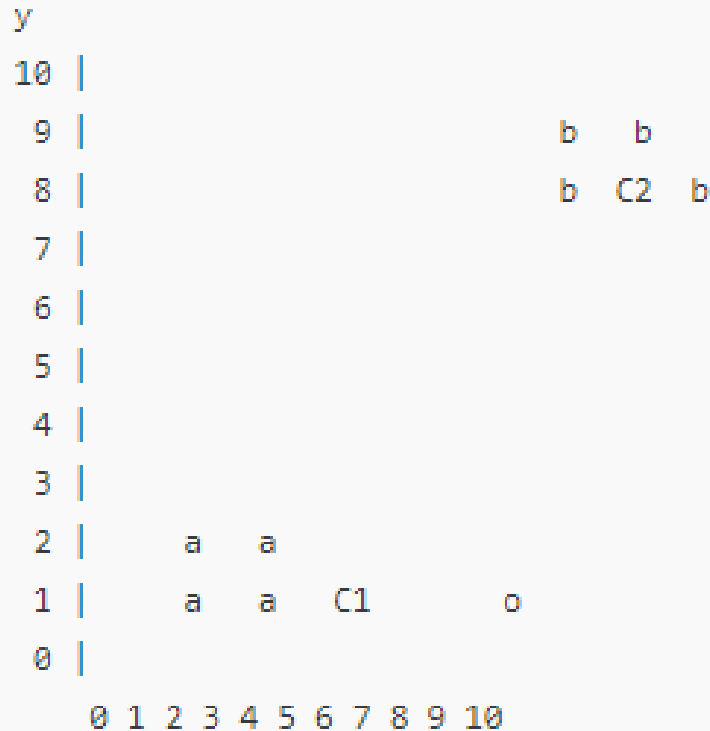
Intuición rápida

- Si te importa un "centro" matemático y rapidez: K-Means
- Si te importa que el centro sea un ejemplo real (interpretabilidad) y resistir outliers: K-Medoids

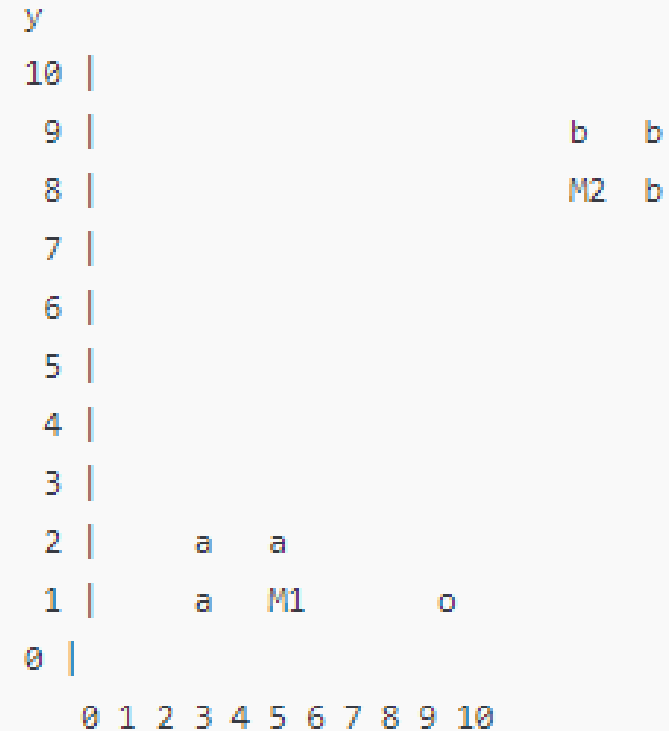
K-Medoids: ejemplo



K-MEANS (centroides = promedios)

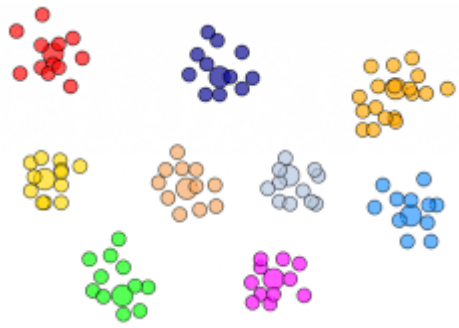


K-MEDOIDS (medoides = puntos reales)

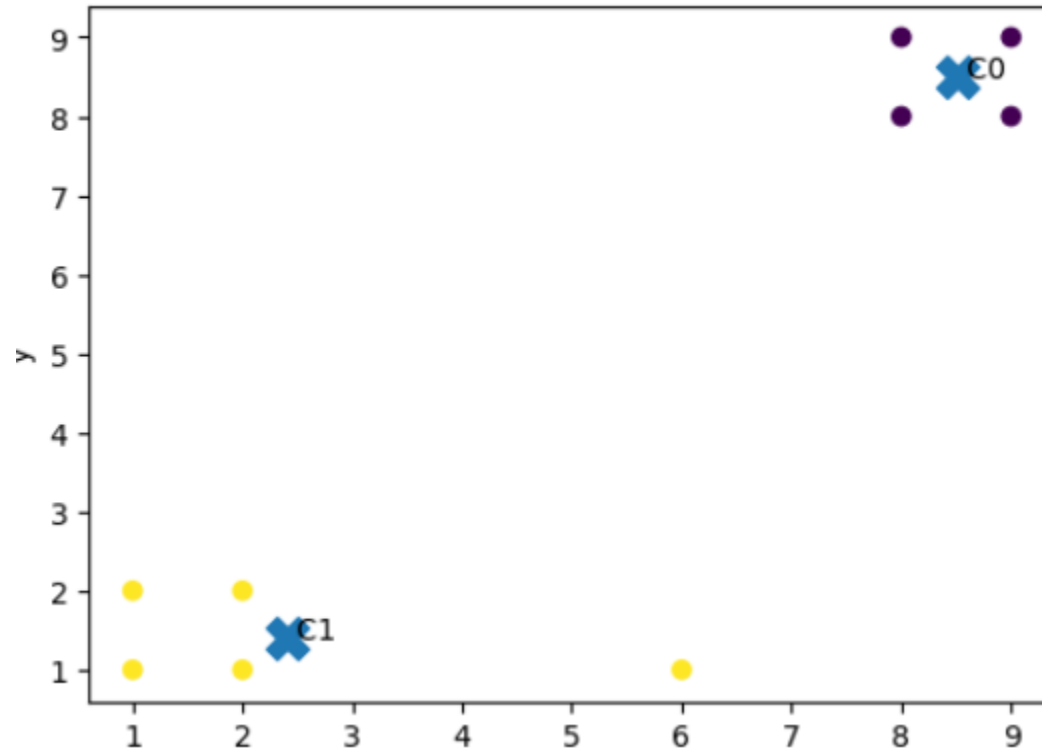


- En K-Means: $C1 \approx (2.4, 1.4)$ y $C2 \approx (8.5, 8.5)$ (los coloco en las celdas más cercanas).
- En K-Medoids: $M1 = (2, 1)$ y $M2 = (8, 8)$ (puntos reales del dataset).

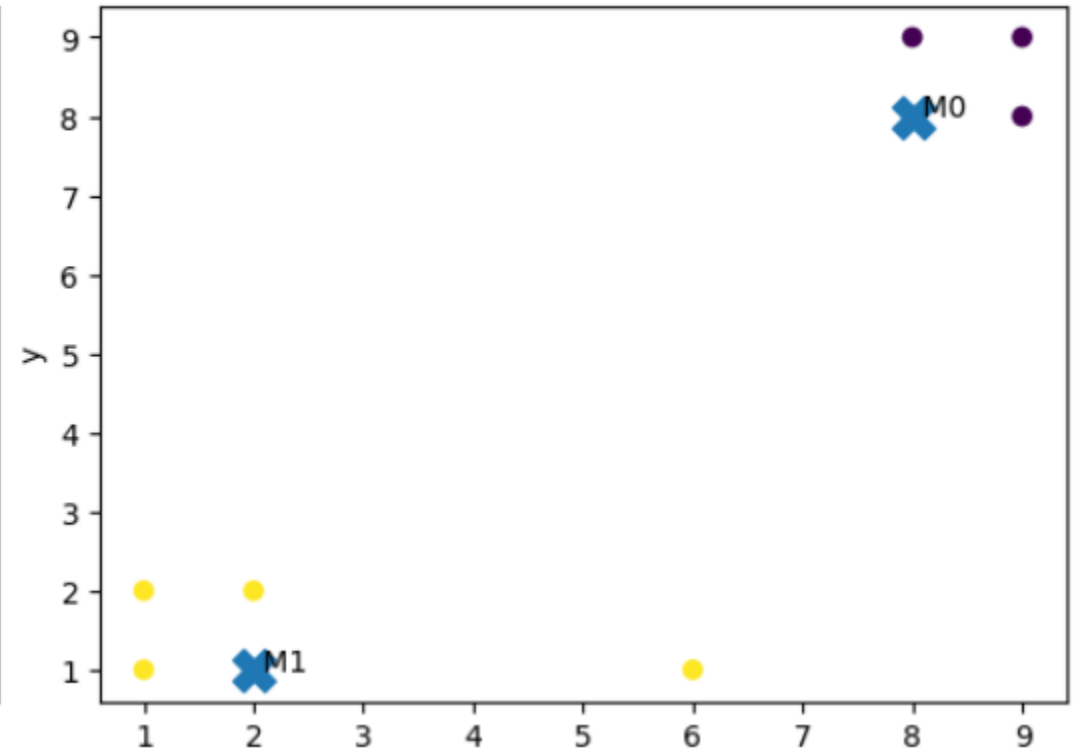
K-Medoids: ejemplo

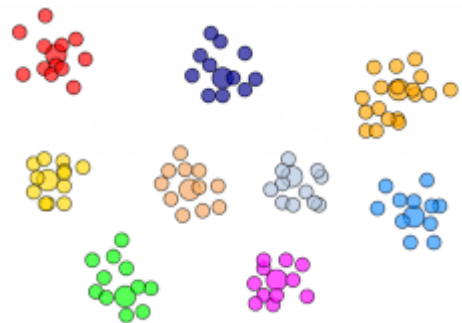


K-Means (centroides)



K-Medoids (medoides)





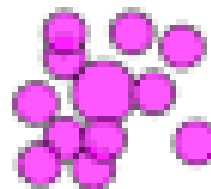
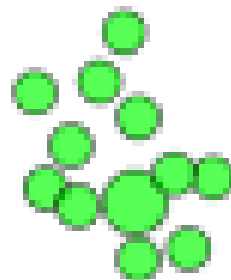
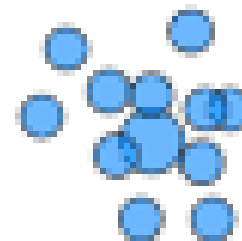
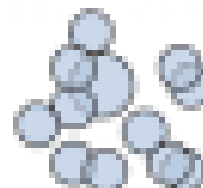
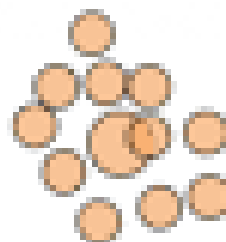
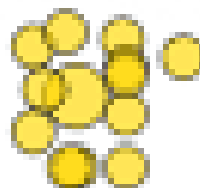
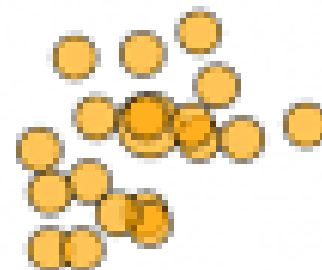
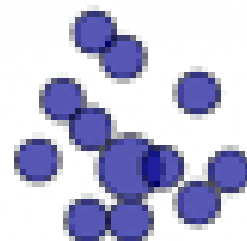
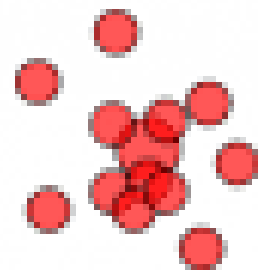
K-Medoids

```
pip install scikit-learn-extra
```

```
from sklearn_extra.cluster import KMedoids

kmed = KMedoids(n_clusters=2, metric="euclidean", method="pam", random_state=0)
labels = kmed.fit_predict(X)
medoid_indices = kmed.medoid_indices_
```

Nota práctica: K-Medoids suele requerir (explícita o implícitamente) una matriz de distancias y puede volverse caro en memoria/tiempo en datasets grandes (comparado con K-Means).



Gracias